

Développement d'un Système de Recommandation Intelligent pour E-commerce

Architecture de Données Distribuées avec PySpark et MinIO

4IASD

30 mars 2026

Table des matières

1	Informations Générales	2
2	Objectifs Pédagogiques	2
3	Dataset et Données	2
3.1	Source de données Proposée	2
3.2	Collecte des données	2
3.3	Prétraitement et Annotation	3
4	Architecture du Système	3
4.1	Pipeline Machine Learning	3
4.2	Stack Technique Détaillée	3
5	Scope du Projet	3
5.1	Fonctionnalités Obligatoires	4
5.2	Fonctionnalités Bonus	4
6	Proposition de Planning en 8 Semaines	4
6.1	Phase 1 : Exploration & Préparation (Semaines 1-2)	4
6.2	Phase 2 : Modélisation & Évaluation (Semaines 3-5)	4
6.3	Phase 3 : Déploiement & Finalisation (Semaines 6-8)	4
7	Répartition des Rôles	4
8	Livrables Attendus	5

1 Informations Générales

Durée	8 semaines
Équipe	2 étudiants par groupe
Stack Technique	Python, PySpark, MLlib, NumPy, SQL, Flask/FastAPI, MinIO
Dataset recommandé	Retailrocket Recommender System Dataset (Kaggle)

Information

Contexte métier : Les plateformes e-commerce exploitent des volumes massifs de données utilisateurs. Ce projet vise à développer un système de recommandation scalable basé sur des technologies Big Data, en utilisant PySpark pour le traitement distribué et MinIO comme stockage objet compatible S3.

2 Objectifs Pédagogiques

Machine Learning Implémenter un moteur de recommandation avec MLlib (ALS)

Big Data Manipuler des données massives avec PySpark

Stockage Utiliser MinIO (Object Storage) pour gérer les données

Prétraitement Nettoyage distribué des données

API Développer un service de recommandation

Analyse Visualiser les comportements utilisateurs

Travail collaboratif Gestion du projet avec Git

3 Dataset et Données

3.1 Source de données Proposée

Dataset

Retailrocket Recommender System Dataset :

- events.csv : interactions utilisateurs-produits
- item_properties.csv : propriétés des produits
- category_tree.csv : catégories
- Volume : plusieurs millions d'interactions

3.2 Collecte des données

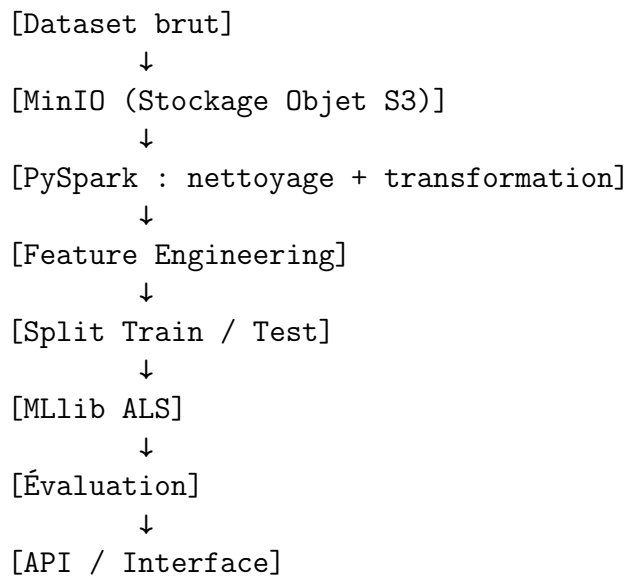
- Téléchargement depuis Kaggle
- Stockage dans MinIO (Object Storage S3)
- Chargement via PySpark

3.3 Prétraitement et Annotation

- Nettoyage distribué avec PySpark
- Construction matrice utilisateur $\times$ produit
- Pondération des interactions (view < cart < achat)

4 Architecture du Système

4.1 Pipeline Machine Learning



4.2 Stack Technique Détaillée

Couche	Technologie	Usage
Langage	Python	Développement principal
Big Data	PySpark	Traitement distribué
ML	Spark MLlib	Filtrage collaboratif (ALS)
Stockage	MinIO	Object Storage compatible S3
Base	PostgreSQL / MongoDB	Données applicatives
API	Flask / FastAPI	Consultation recommandations
Versioning	Git	Gestion du code

5 Scope du Projet

5.1 Fonctionnalités Obligatoires

- Prétraitement distribué avec PySpark
- Implémentation ALS (MLlib)
- Stockage des données avec MinIO
- API de recommandation
- Visualisation des résultats
- Documentation complète

5.2 Fonctionnalités Bonus

- Déploiement cloud
- Dashboard interactif
- Modèle hybride
- Analyse des tendances produits

6 Proposition de Planning en 8 Semaines

6.1 Phase 1 : Exploration & Préparation (Semaines 1-2)

S1	Collecte et stockage dans MinIO	Données brutes
S2	Prétraitement avec PySpark	Dataset prêt

6.2 Phase 2 : Modélisation & Évaluation (Semaines 3-5)

S3	Entraînement ALS	Modèle initial
S4	Évaluation (RMSE, Precision@K)	Rapport
S5	Optimisation	Modèle final

6.3 Phase 3 : Déploiement & Finalisation (Semaines 6-8)

S6	API	Application fonctionnelle
S7	Visualisation	Dashboard
S8	Rapport	Soutenance

7 Répartition des Rôles

E1	Data Engineer	PySpark, MinIO, pipeline Big Data
E2	ML/Backend	MLlib, API, visualisation

Information

Les deux étudiants doivent comprendre l'ensemble du pipeline Big Data pour la soutenance.

8 Livrables Attendus

1. Code source (PySpark + MLlib + MinIO)
2. API ou interface fonctionnelle
3. Rapport technique détaillé

Résumé

Ce projet consiste à développer un système de recommandation Big Data scalable capable de traiter de grandes quantités de données utilisateurs. Grâce à l'utilisation de PySpark pour le traitement distribué et MinIO comme stockage objet, le système permet de générer des recommandations personnalisées de manière efficace et performante.